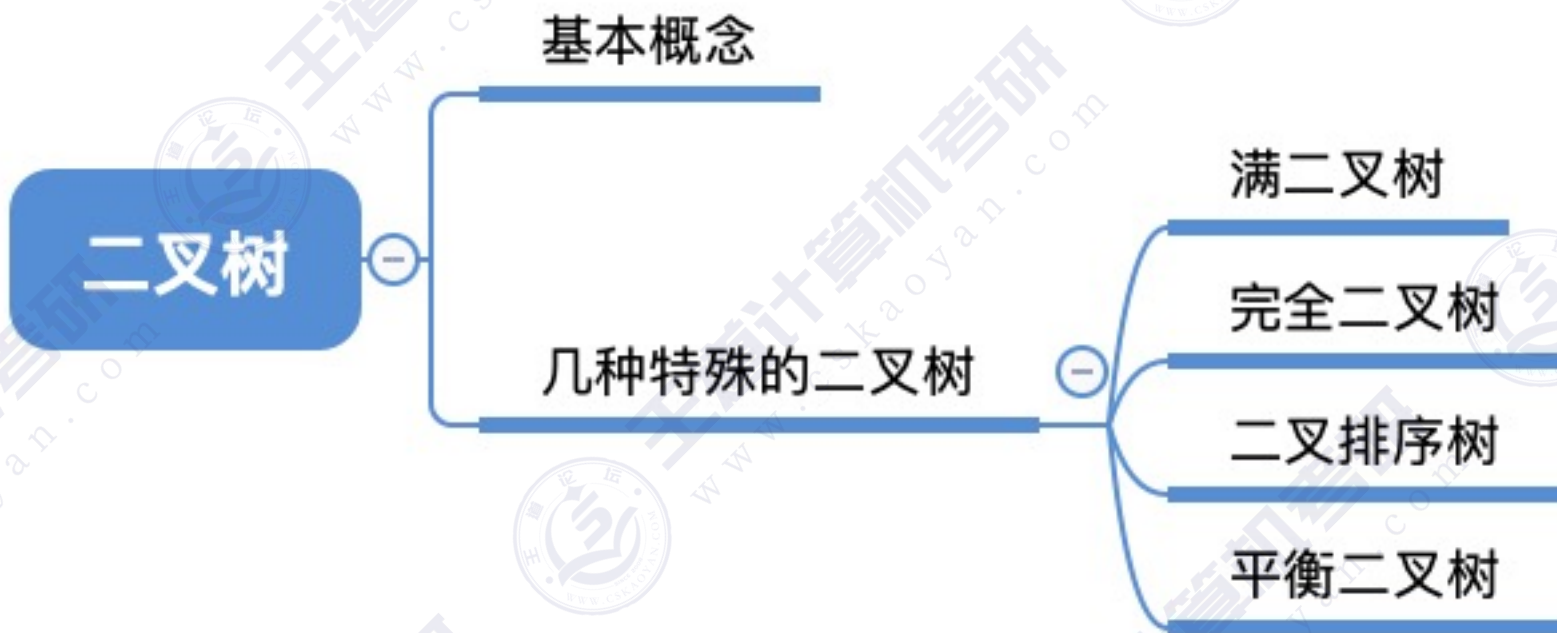


本节内容

二叉树

定义
基本术语

知识总览



二叉树的基本概念

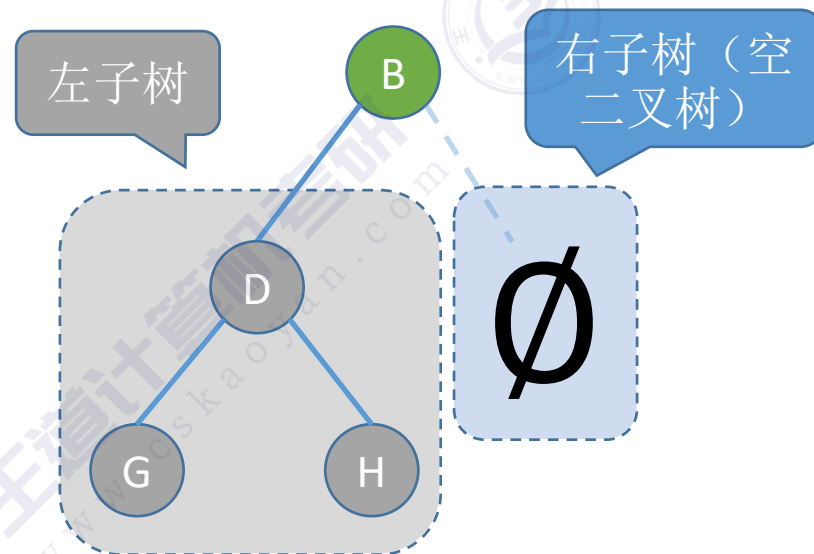
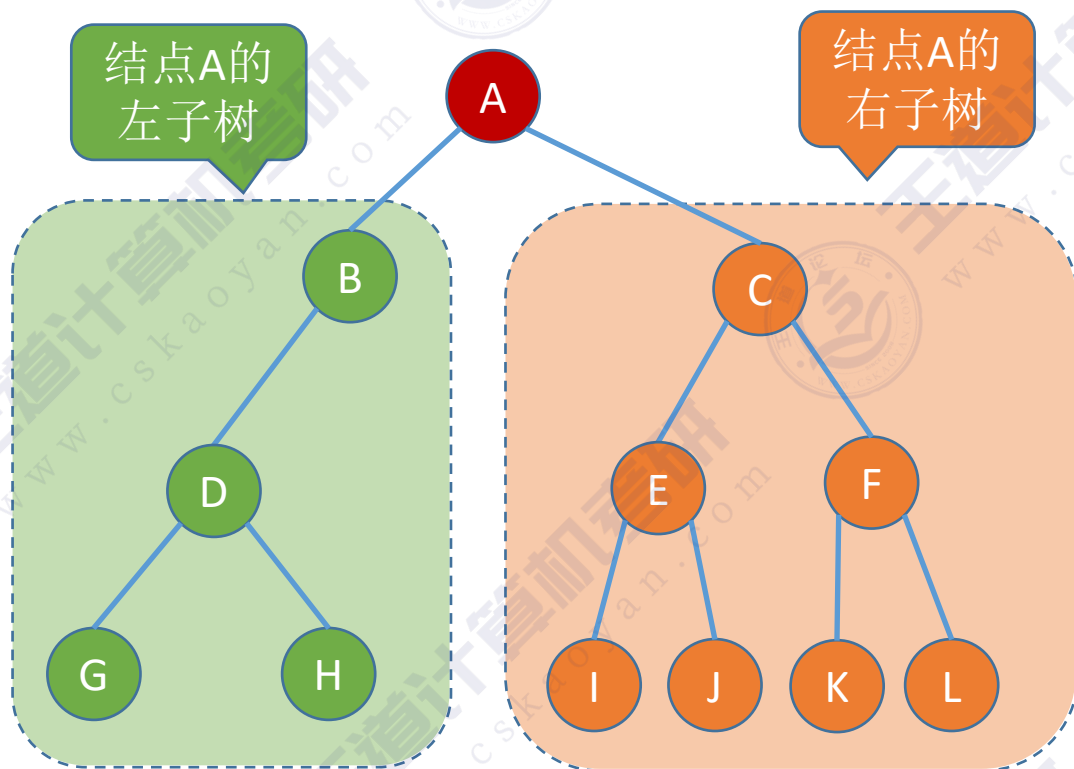
二叉树是 n ($n \geq 0$) 个结点的有限集合:

① 或者为**空二叉树**, 即 $n = 0$ 。

② 或者由一个**根结点**和两个互不相交的被称为根的**左子树**和**右子树**组成。左子树和右子树又分别是一棵二叉树。

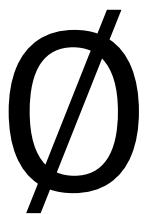
特点: ①每个结点至多只有两棵子树 ②左右子树不能颠倒 (二叉树是**有序树**)

注意区别: 度为2的有序树

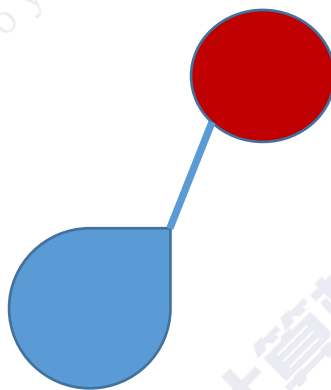


🌲 二叉树是递归定义的数据结构

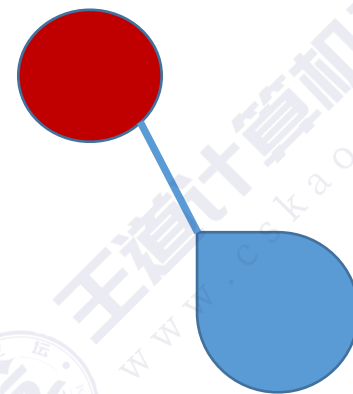
二叉树的五种状态



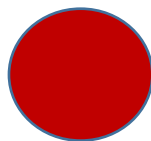
空二叉树



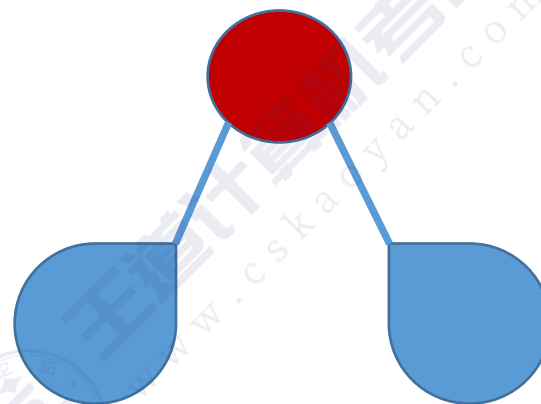
只有左子树



只有右子树



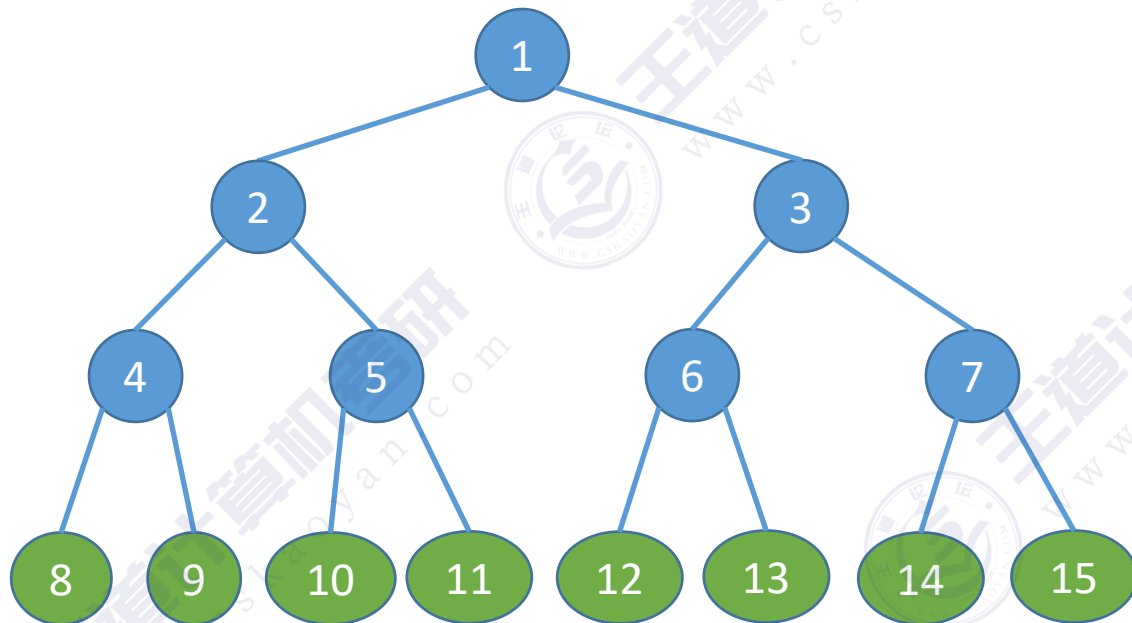
只有根节点



左右子树都有

几个特殊的二叉树

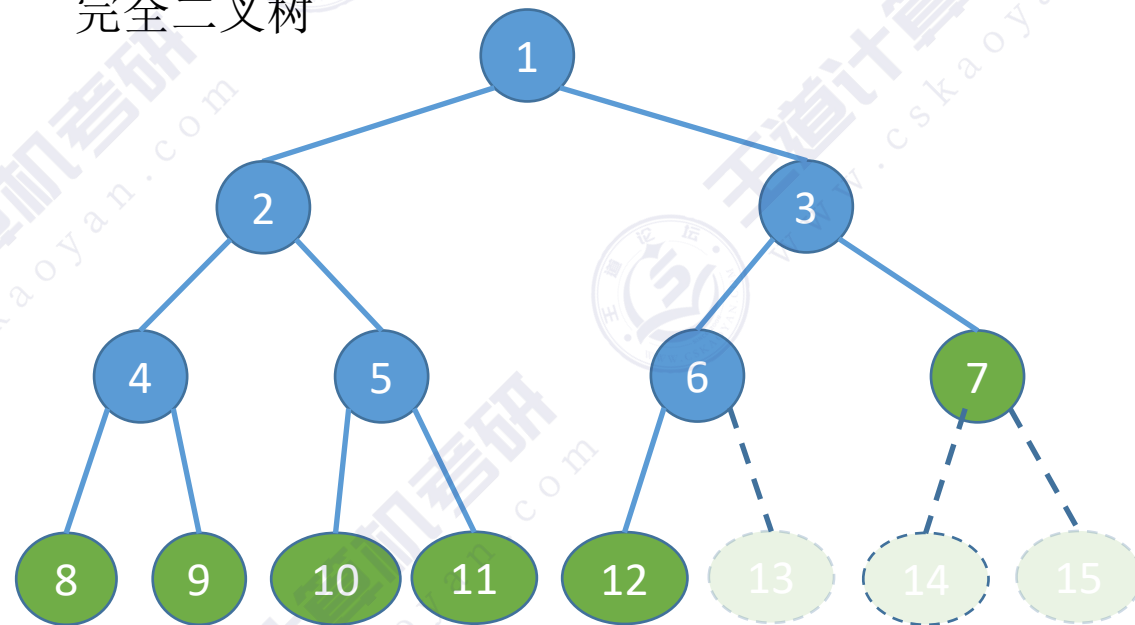
满二叉树。一棵高度为 h ，且含有 $2^h - 1$ 个结点的二叉树



特点:

- ①只有最后一层有叶子结点
- ②不存在度为1的结点
- ③按层序从1开始编号，结点 i 的左孩子为 $2i$ ，右孩子为 $2i+1$ ；结点 i 的父节点为 $\lfloor i/2 \rfloor$ （如果有的话）

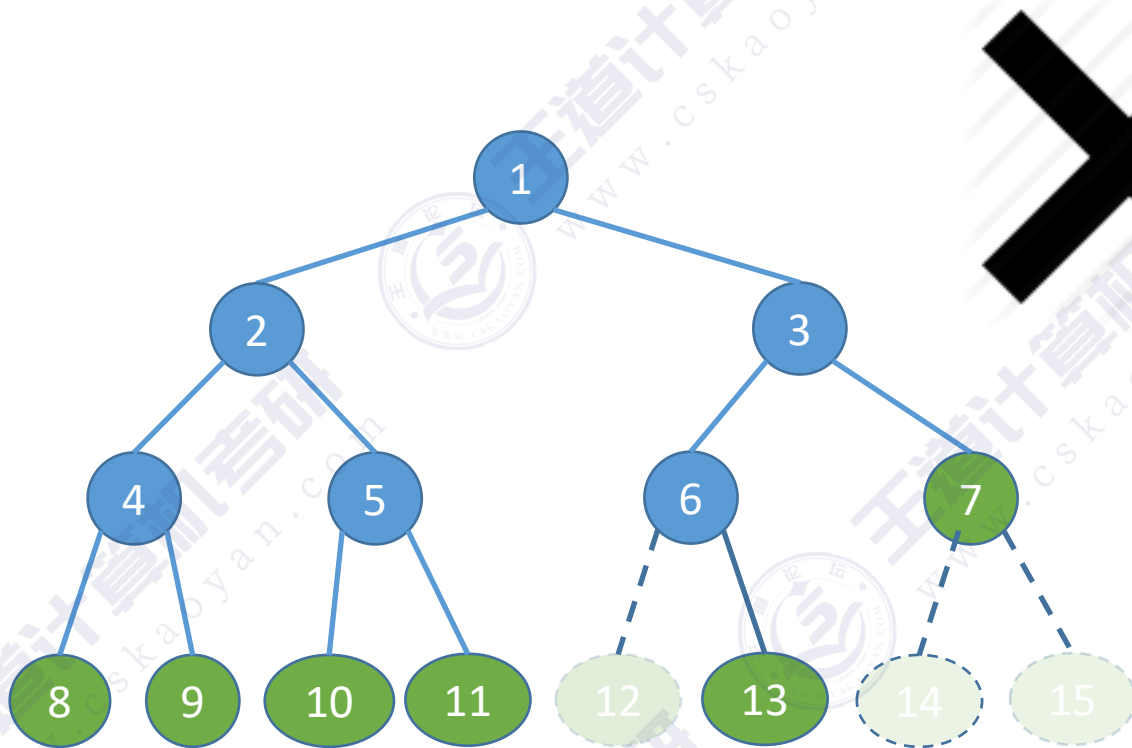
完全二叉树。当且仅当其每个结点都与高度为 h 的满二叉树中编号为 $1 \sim n$ 的结点一一对应时，称为完全二叉树



特点:

- ①只有最后两层可能有叶子结点
- ②最多只有一个度为1的结点
- ③同左③
- ④ $i \leq \lfloor n/2 \rfloor$ 为分支结点， $i > \lfloor n/2 \rfloor$ 为叶子结点

几个特殊的二叉树



不是“完全二叉树”

如果某结点只有一个孩子，
那么一定是左孩子



几个特殊的二叉树

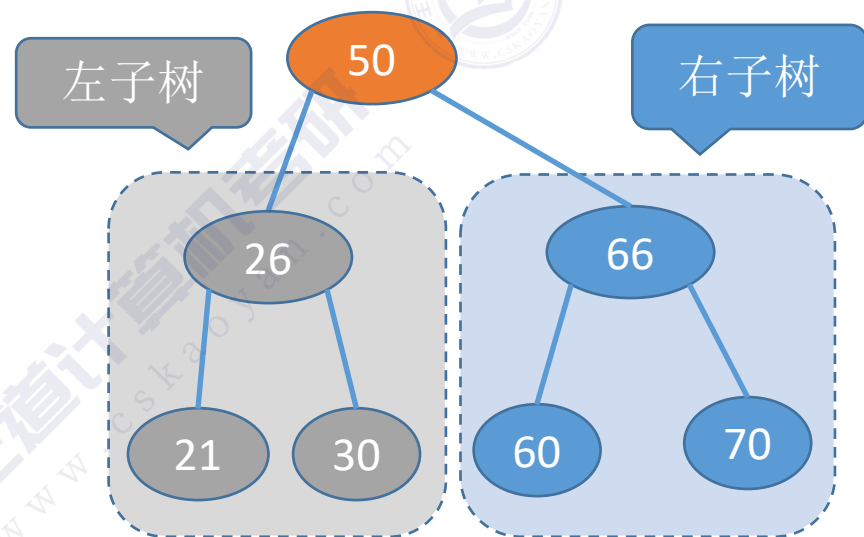
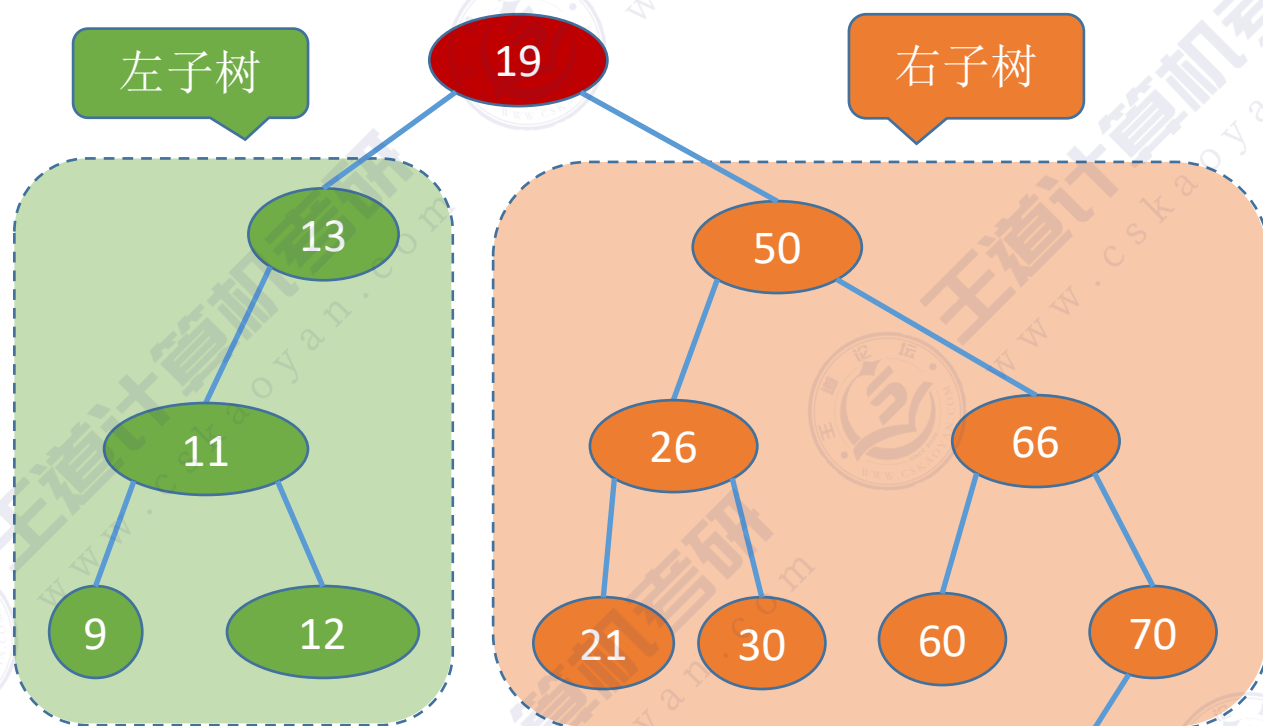
二叉排序树。一棵二叉树或者是空二叉树，或者是具有如下性质的二叉树：

左子树上所有结点的**关键字**均**小于根结点**的关键字；

右子树上所有结点的**关键字**均**大于根结点**的关键字。

左子树和右子树又各是一棵二叉排序树。

二叉排序树可用于元素的排序、搜索

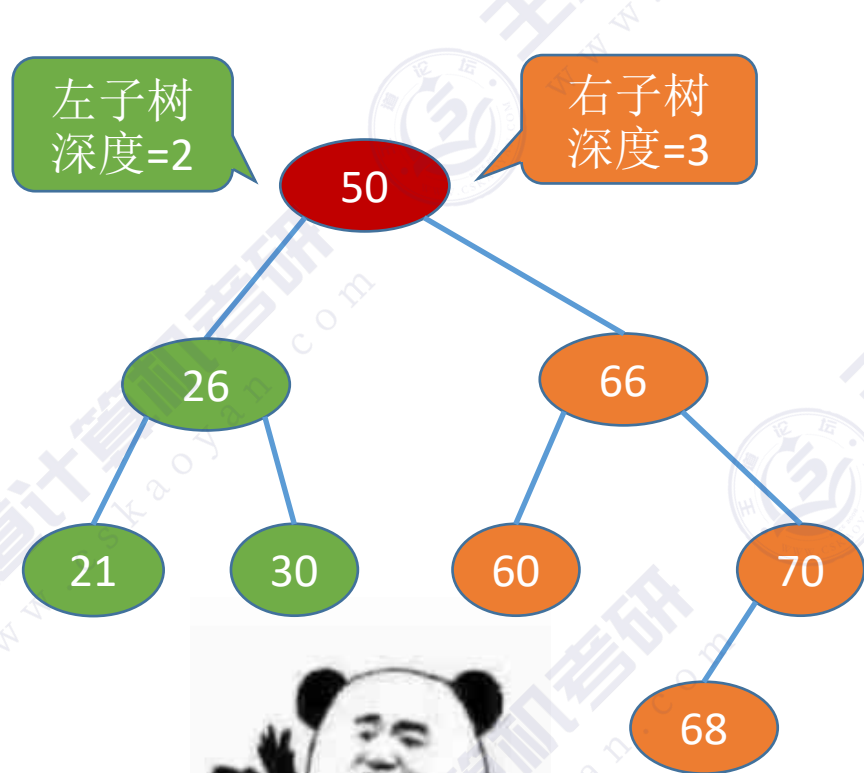


几个特殊的二叉树

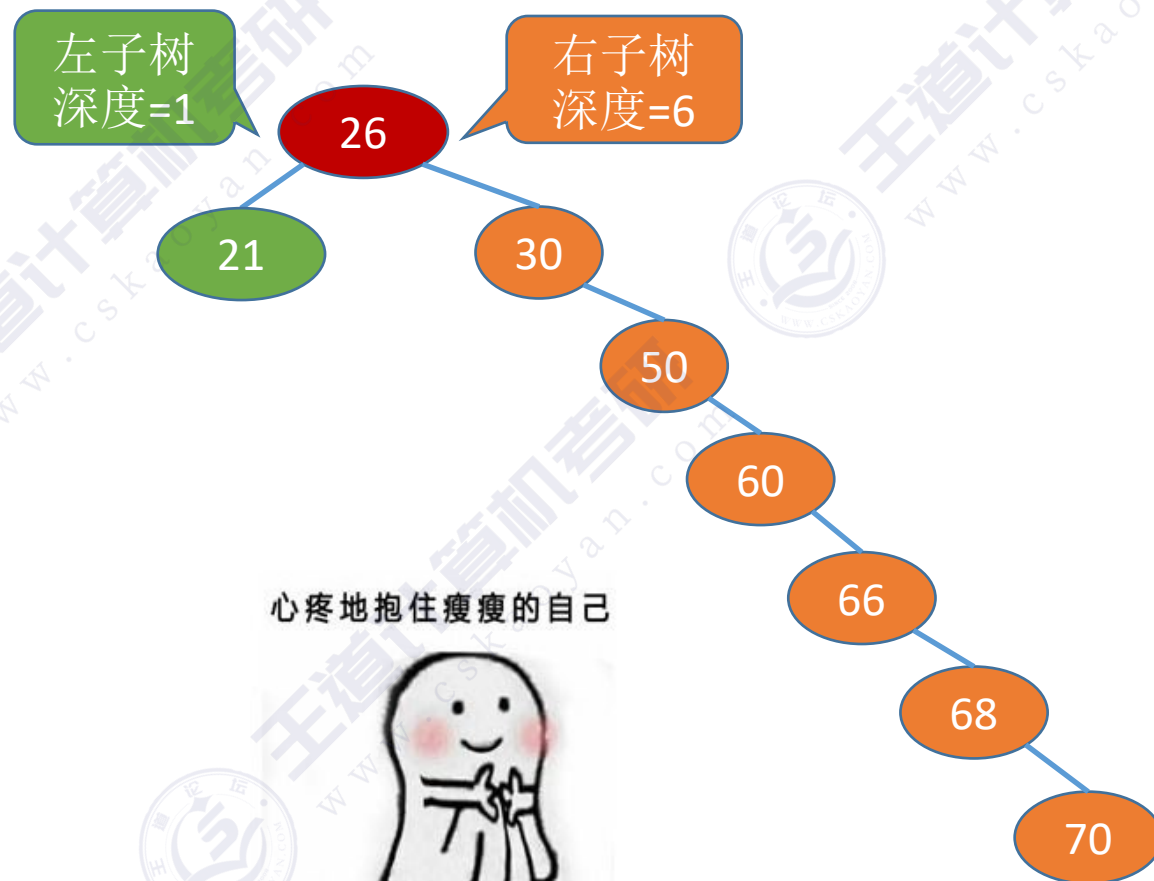
胖胖的、丰满的树

平衡二叉树。树上任一结点的左子树和右子树的深度之差不超过1。

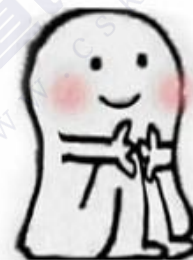
平衡二叉树能有更高的搜索效率



那你很胖胖哦



心疼地抱住瘦瘦的自己



知识回顾与重要考点

二叉树

基本概念

可为空二叉树

任意结点的度 ≤ 2

是有序树，左子树、右子树不可颠倒

思考：二叉树 V.S. 度为2的有序树

特殊二叉树

满二叉树

高度为 h ，含有 $2^h - 1$ 个结点的二叉树

完全二叉树

在满二叉树的基础上可去掉若干个编号更大的结点

二叉排序树

左子树关键字 $<$ 根节点关键字 $<$ 右子树关键字

平衡二叉树

左右子树深度差不超过1

本节内容

二叉树

常考性质

二叉树的常考性质

常见考点1：设非空二叉树中度为0、1和2的结点个数分别为 n_0 、 n_1 和 n_2 ，则 $n_0 = n_2 + 1$
(叶子结点比二分支结点多一个)

假设树中结点总数为 n ，则

① $n = n_0 + n_1 + n_2$

② $n = n_1 + 2n_2 + 1$

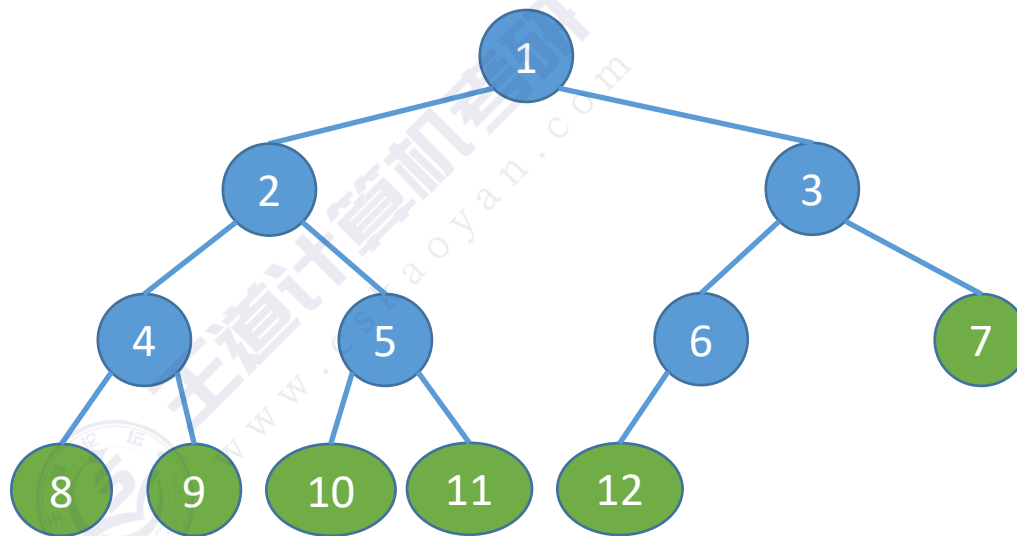
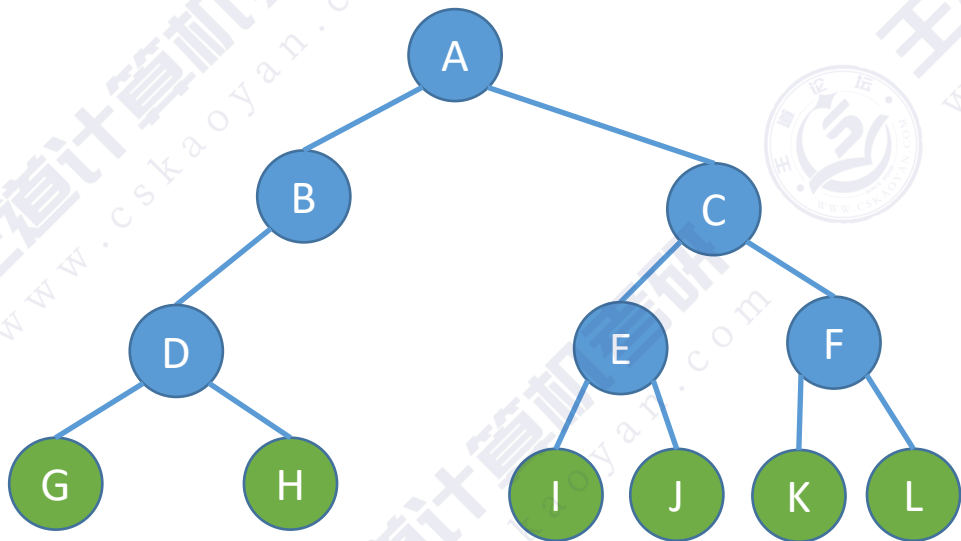
② - ①

$$n_0 = n_2 + 1$$



上面可都是重点

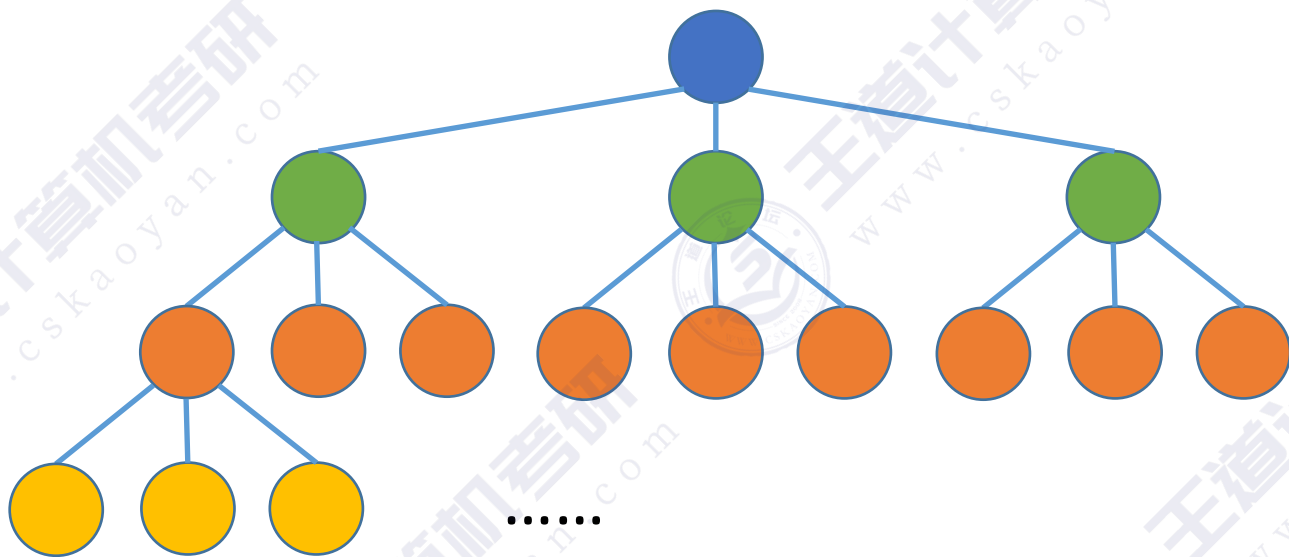
树的结点数=总度数+1



二叉树的常考性质

常见考点2: 二叉树第 i 层至多有 2^{i-1} 个结点 ($i \geq 1$)

m 叉树第 i 层至多有 m^{i-1} 个结点 ($i \geq 1$)



第 1 层: m^0

第 2 层: m^1

第 3 层: m^2

第 4 层: m^3

二叉树的常考性质

常见考点3: 高度为 h 的二叉树至多有 $2^h - 1$ 个结点 (满二叉树)

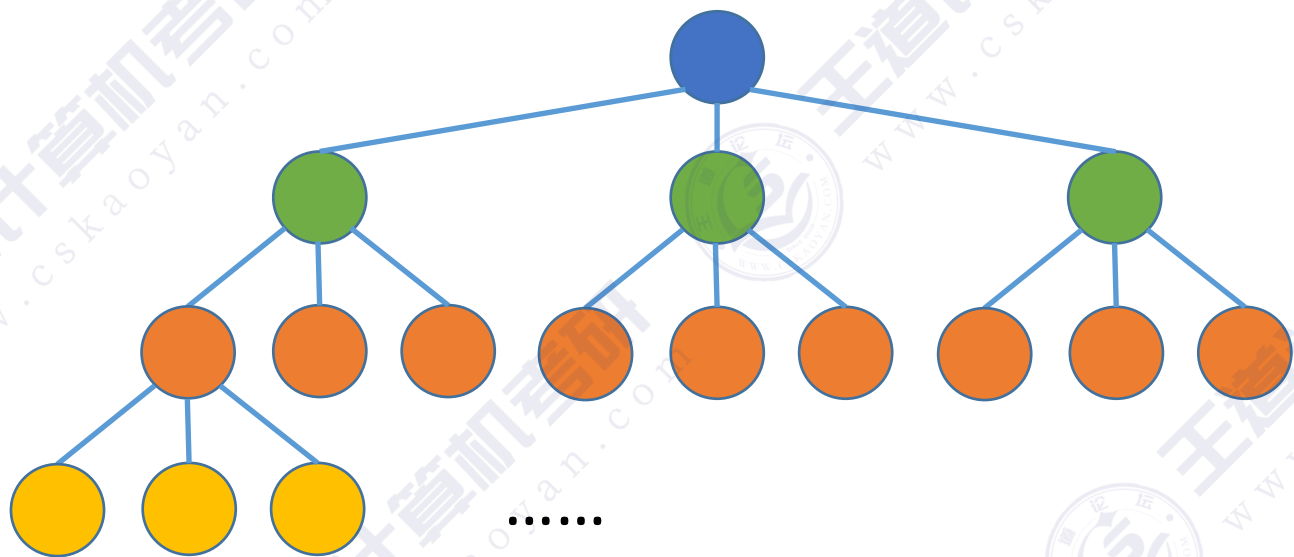
高度为 h 的 m 叉树至多有 $\frac{m^h - 1}{m - 1}$ 个结点

等比数列求和公式: $a + aq + aq^2 + \dots + aq^{n-1} = \frac{a(1 - q^n)}{1 - q}$

至少有多少个?



思考考



第1层: m^0

第2层: m^1

第3层: m^2

第4层: m^3

完全二叉树的常考性质

常见考点1: 具有 n 个 ($n > 0$) 结点的完全二叉树的高度 h 为 $\lceil \log_2(n+1) \rceil$ 或 $\lfloor \log_2 n \rfloor + 1$

高为 h 的满二叉树共有 $2^h - 1$ 个结点
高为 $h-1$ 的满二叉树共有 $2^{h-1} - 1$ 个结点

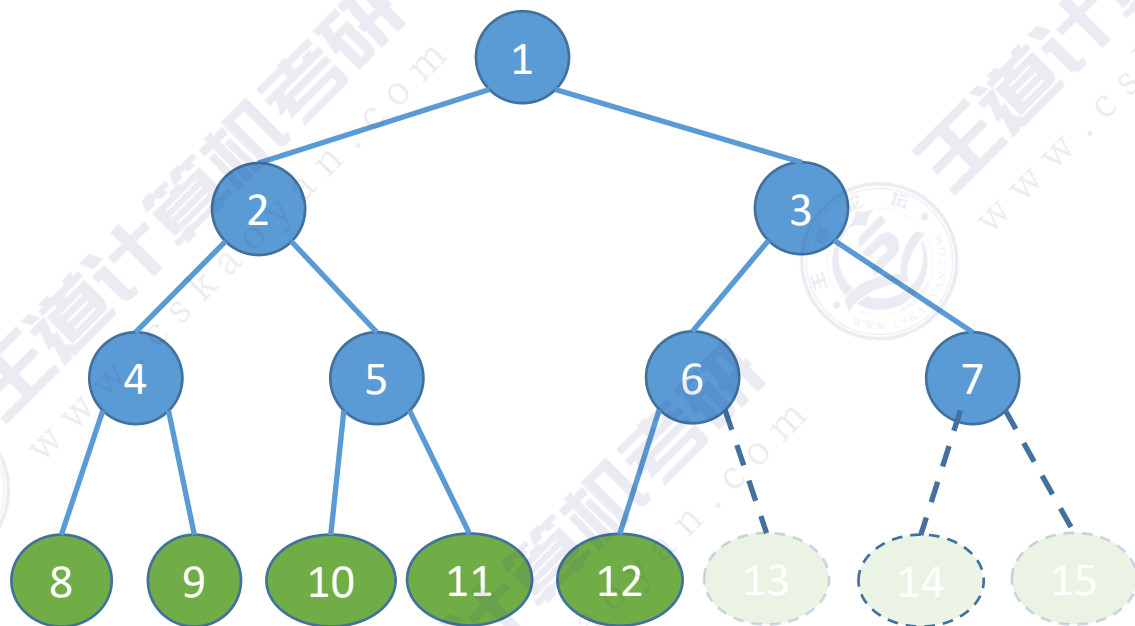


$$2^{h-1} - 1 < n \leq 2^h - 1$$

$$2^{h-1} < n + 1 \leq 2^h$$

$$h - 1 < \log_2(n + 1) \leq h$$

$$h = \lceil \log_2(n + 1) \rceil$$



完全二叉树的常考性质

常见考点1: 具有 n 个 ($n > 0$) 结点的完全二叉树的高度 h 为 $\lceil \log_2(n+1) \rceil$ 或 $\lfloor \log_2 n \rfloor + 1$

高为 $h-1$ 的满二叉树共有 $2^{h-1} - 1$ 个结点

高为 h 的完全二叉树至少 2^{h-1} 个结点

至多 $2^h - 1$ 个结点



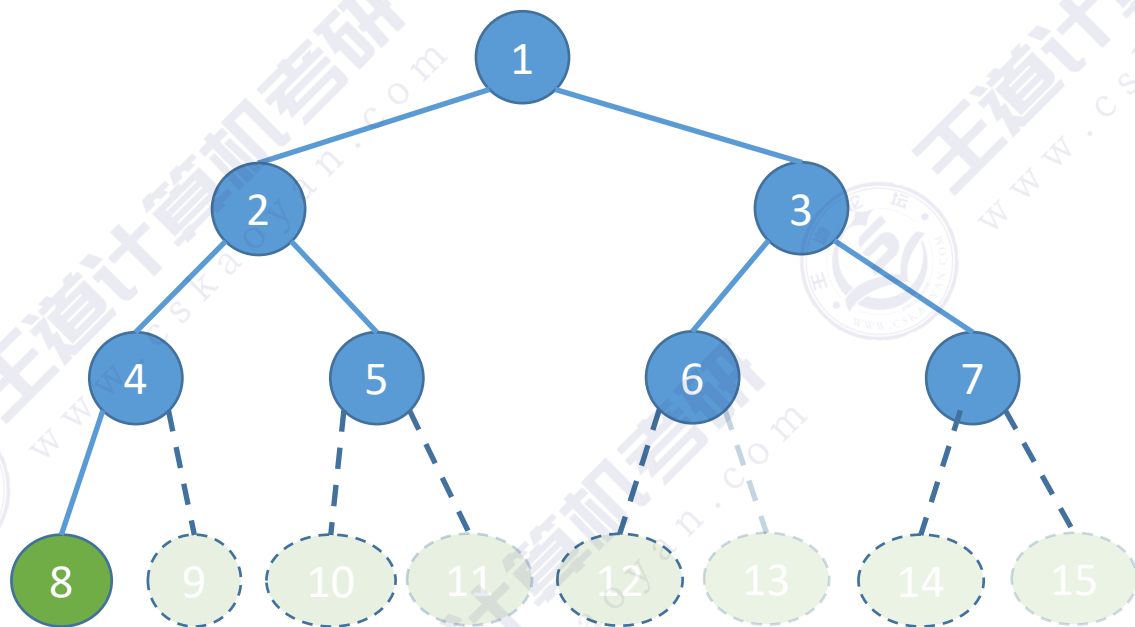
$$2^{h-1} \leq n < 2^h$$

$$h - 1 \leq \log_2 n < h$$

$$h = \lfloor \log_2 n \rfloor + 1$$



第 i 个结点所在层次为 $\lceil \log_2(n+1) \rceil$ 或 $\lfloor \log_2 n \rfloor + 1$



完全二叉树的常考性质

常见考点2：对于完全二叉树，可以由的结点数 n 推出度为0、1和2的结点个数为 n_0 、 n_1 和 n_2

完全二叉树最多只有一个度为1的结点，即

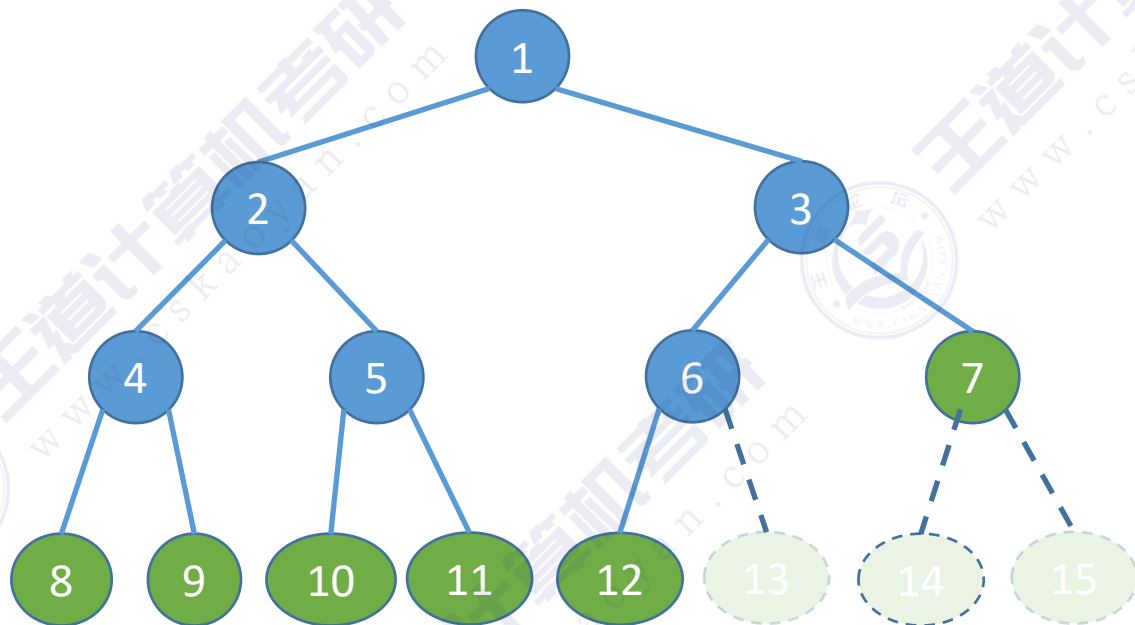
$n_1=0$ 或1

$n_0 = n_2 + 1 \rightarrow n_0 + n_2$ 一定是奇数



若完全二叉树有 $2k$ 个（偶数）个结点，则必有 $n_1=1$, $n_0=k$, $n_2=k-1$

若完全二叉树有 $2k-1$ 个（奇数）个结点，则必有 $n_1=0$, $n_0=k$, $n_2=k-1$



知识回顾与重要考点

二叉树：

- $n_0 = n_2 + 1$
- 第 i 层至多有 2^{i-1} 个结点 ($i \geq 1$)
- 高度为 h 的二叉树至多有 $2^h - 1$ 个结点

完全二叉树：

- 具有 n 个 ($n > 0$) 结点的完全二叉树的高度 h 为 $\lceil \log_2(n+1) \rceil$ 或 $\lfloor \log_2 n \rfloor + 1$
- 对于完全二叉树，可以由的结点数 n 推出为0、1和2的结点个数分别为 n_0 、 n_1 和 n_2 （突破点：完全二叉树最多只会会有一个度为1的结点）

本节内容

二叉树

存储结构

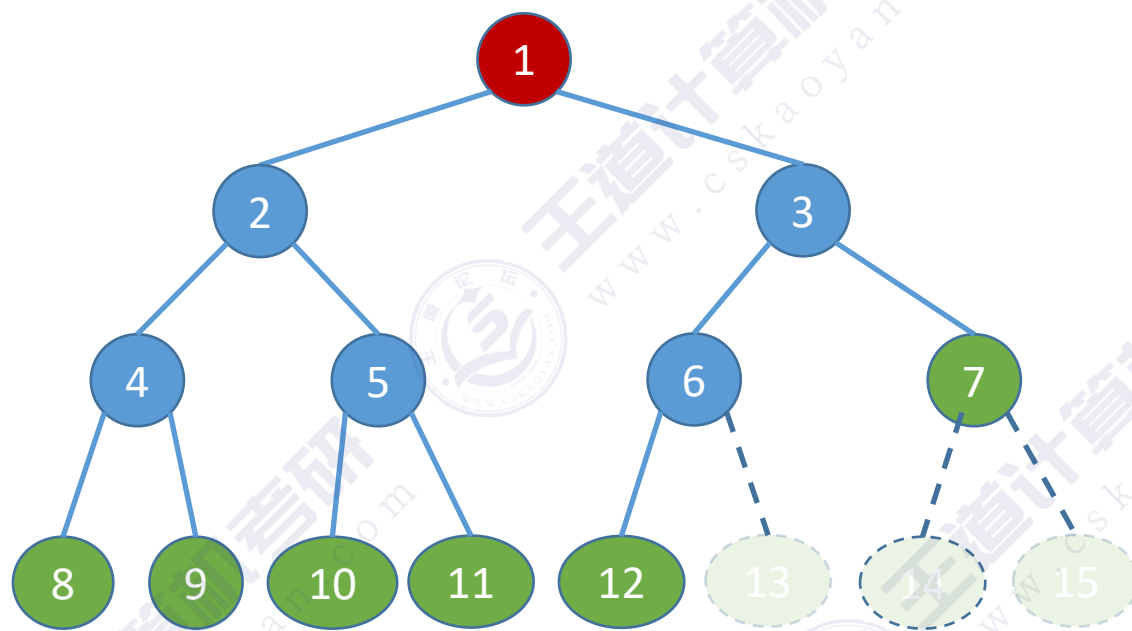
知识总览

二叉树的存储结构

顺序存储

链式存储

二叉树的顺序存储



```
#define MaxSize 100
struct TreeNode {
    ElemType value; // 结点中的数据元素
    bool isEmpty;   // 结点是否为空
};
```

```
TreeNode t[MaxSize];
```

定义一个长度为 `MaxSize` 的数组 `t`，按照从上至下、从左至右的顺序依次存储完全二叉树中的各个结点



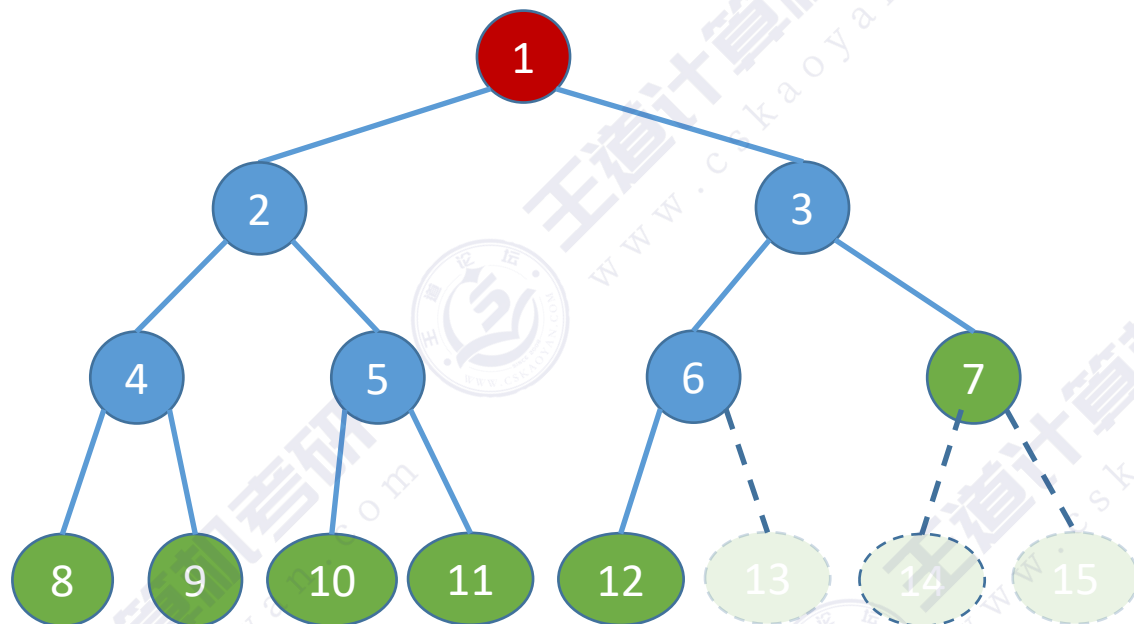
`t[0]` `t[1]` `t[2]`

可以让第一个位置空缺，保证数组下标和结点编号一致

初始化时所有结点标记为空

```
for (int i=0; i<MaxSize; i++){
    t[i].isEmpty=true;
}
```


二叉树的顺序存储



几个重要常考的基本操作:

- i 的左孩子 —— $2i$
- i 的右孩子 —— $2i+1$
- i 的父节点 —— $\lfloor i/2 \rfloor$
- i 所在的层次 —— $\lceil \log_2(n+1) \rceil$ 或 $\lfloor \log_2 n \rfloor + 1$

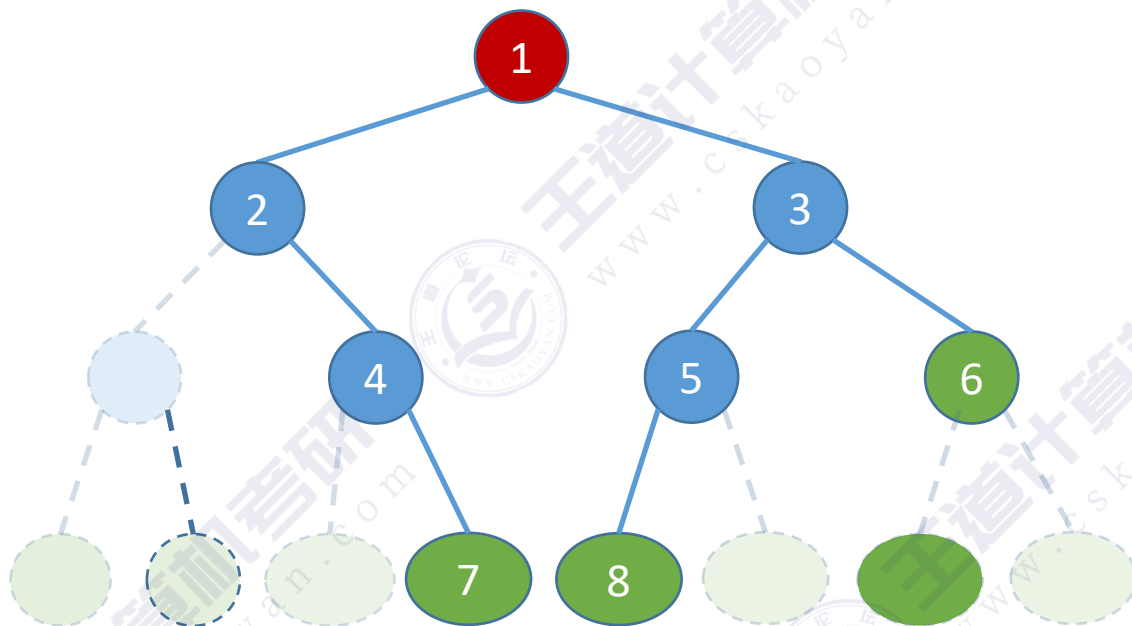
若完全二叉树中共有 n 个结点, 则

- 判断 i 是否有左孩子? —— $2i \leq n$?
- 判断 i 是否有右孩子? —— $2i+1 \leq n$?
- 判断 i 是否是叶子/分支结点? —— $i > \lfloor n/2 \rfloor$?



$t[0]$ $t[1]$ $t[2]$

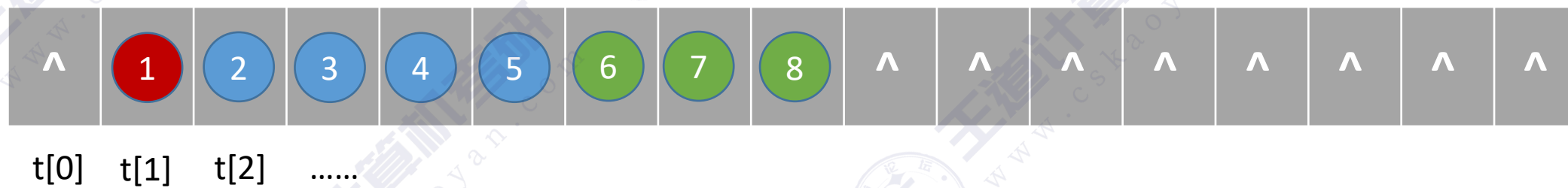
二叉树的顺序存储



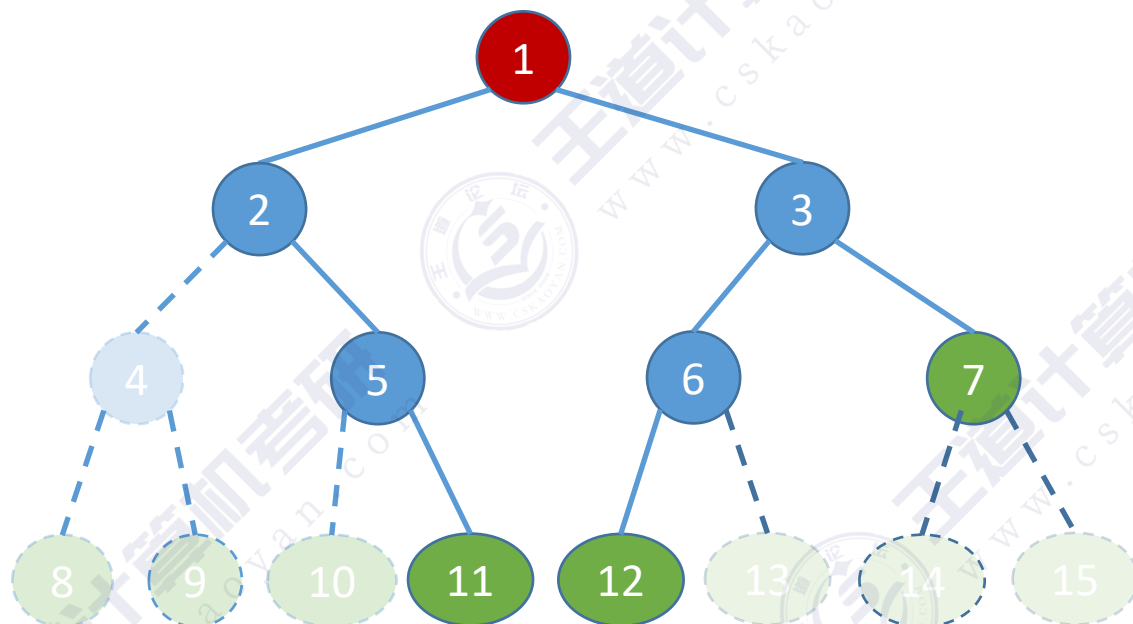
如果不是完全二叉树，
依然按层序将各节点
顺序存储，那么...

无法从结点编号反映
出结点间的逻辑关系

- i 的左孩子 —— $2i$
- i 的右孩子 —— $2i+1$
- i 的父节点 —— $\lfloor i/2 \rfloor$



二叉树的顺序存储



二叉树的顺序存储中，一定要把二叉树的结点编号与完全二叉树对应起来

- i 的左孩子 —— $2i$
- i 的右孩子 —— $2i+1$
- i 的父节点 —— $\lfloor i/2 \rfloor$



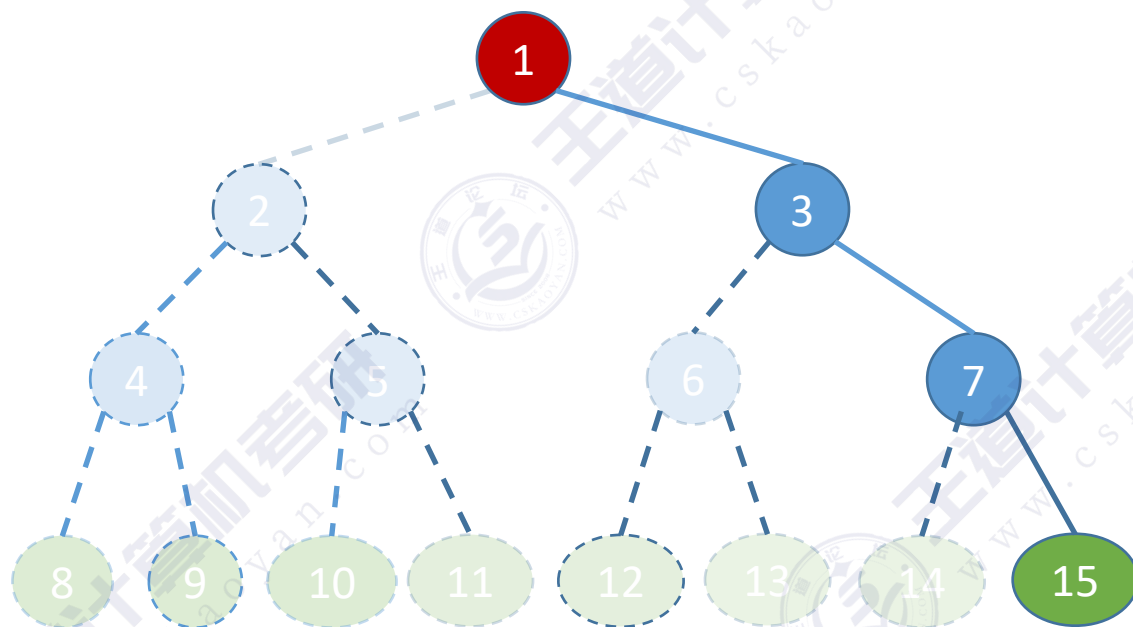
若非完全二叉树中共有 n 个结点，则

- 判断 i 是否有左孩子? —— $2i \leq n$?
- 判断 i 是否有右孩子? —— $2i+1 \leq n$?
- 判断 i 是否是叶子/分支结点? —— $i > \lfloor n/2 \rfloor$?



$t[0]$ $t[1]$ $t[2]$

二叉树的顺序存储



二叉树的顺序存储中，一定要把二叉树的结点编号与完全二叉树对应起来

- i 的左孩子 —— $2i$
- i 的右孩子 —— $2i+1$
- i 的父节点 —— $\lfloor i/2 \rfloor$

最坏情况：高度为 h 且只有 h 个结点的单支树（所有结点只有右孩子），也至少需要 2^h-1 个存储单元

结论：二叉树的顺序存储结构，只适合存储完全二叉树



$t[0]$ $t[1]$ $t[2]$

二叉树的链式存储

//二叉树的结点 (链式存储)

```
typedef struct BiTNode{  
    ElemType data;  
    struct BiTNode *lchild,*rchild;  
}BiTNode,*BiTree;
```

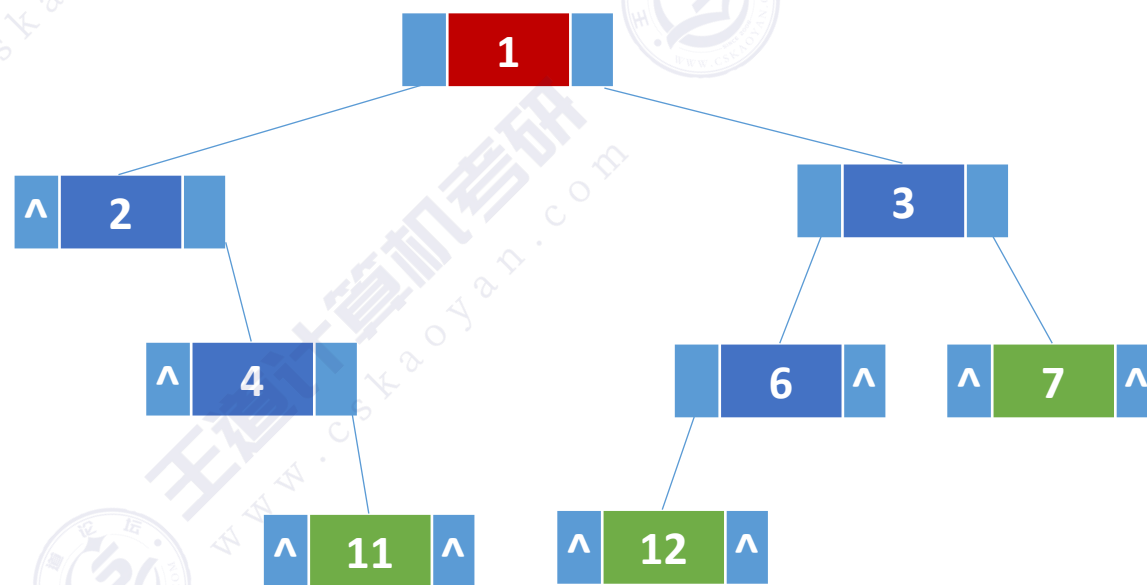
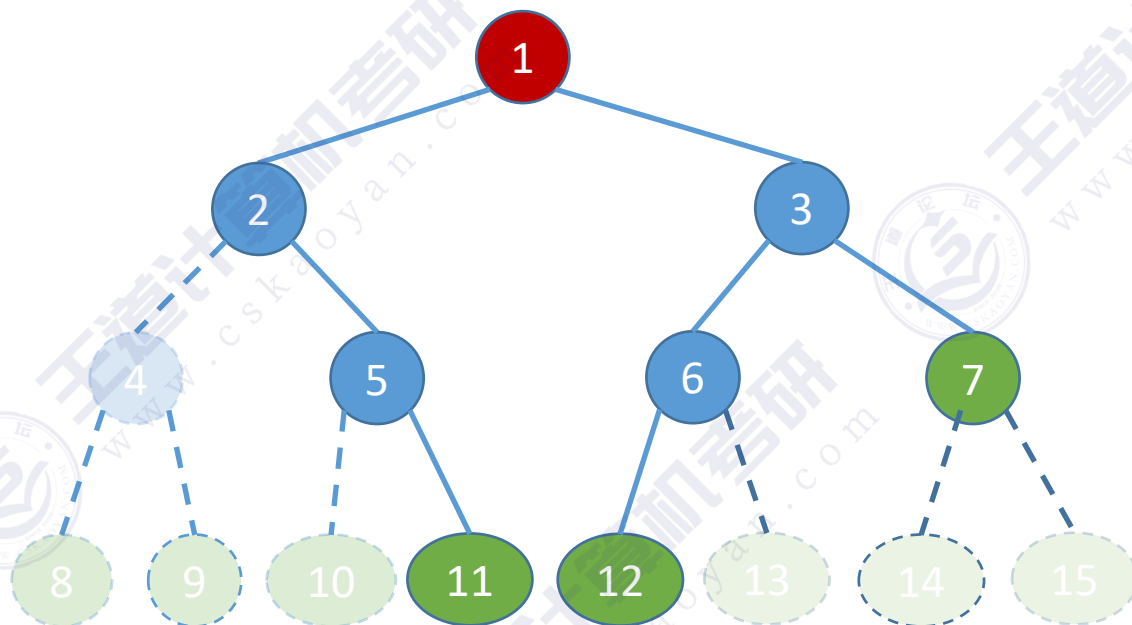
//数据域

//左、右孩子指针

可以用于构造
线索二叉树



n个结点的二叉链表共有 $n+1$ 个空链域



二叉树的链式存储

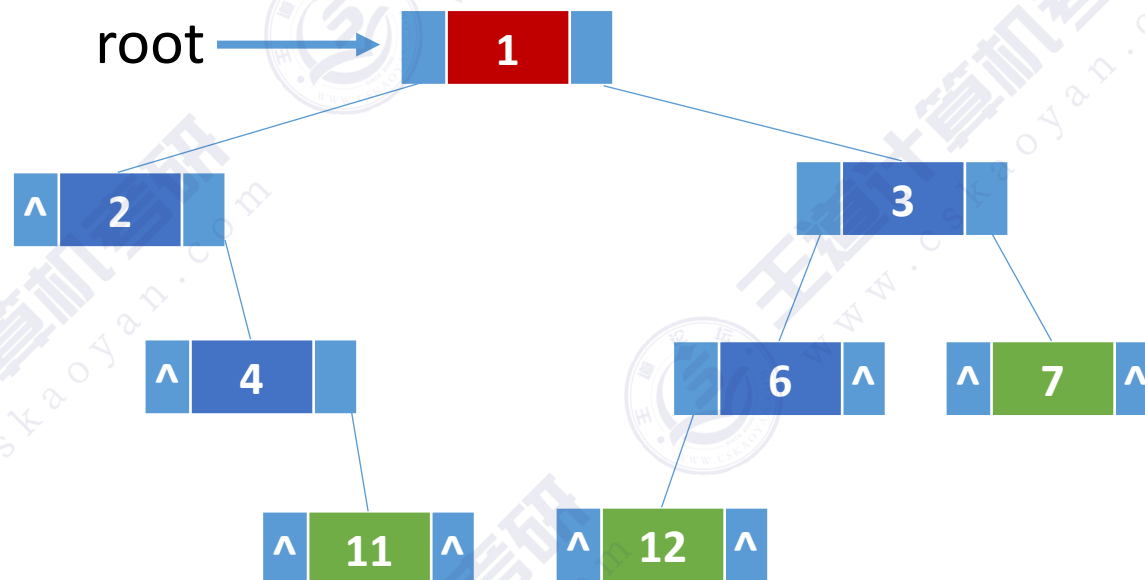
```
struct ElemType{  
    int value;  
};
```

```
typedef struct BiTNode{  
    ElemType data;  
    struct BiTNode *lchild,*rchild;  
}BiTNode,*BiTree;
```

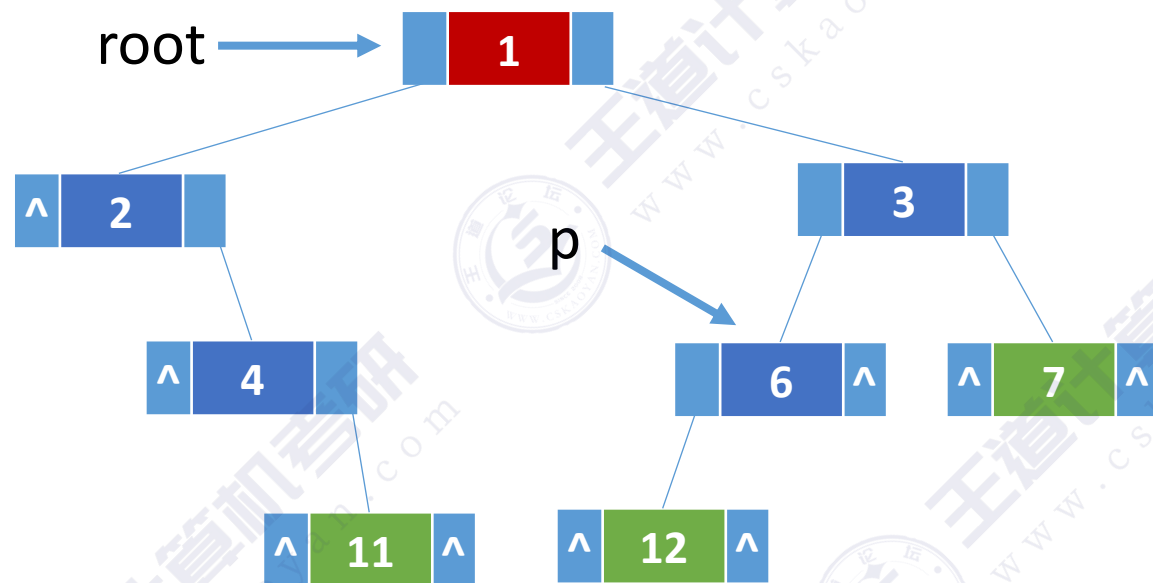
```
//定义一棵空树  
BiTree root = NULL;
```

```
//插入根节点  
root = (BiTree) malloc(sizeof(BiTNode));  
root->data = {1};  
root->lchild = NULL;  
root->rchild = NULL;
```

```
//插入新结点  
BiTNode * p = (BiTNode *) malloc(sizeof(BiTNode));  
p->data = {2};  
p->lchild = NULL;  
p->rchild = NULL;  
root->lchild = p;    //作为根节点的左孩子
```



二叉树的链式存储



找到指定结点 p 的左/右孩子——超简单

如何找到指定结点 p 的父结点？

只能从根开始遍历寻找

Tips: 根据实际需求决定要不要加父结点指针

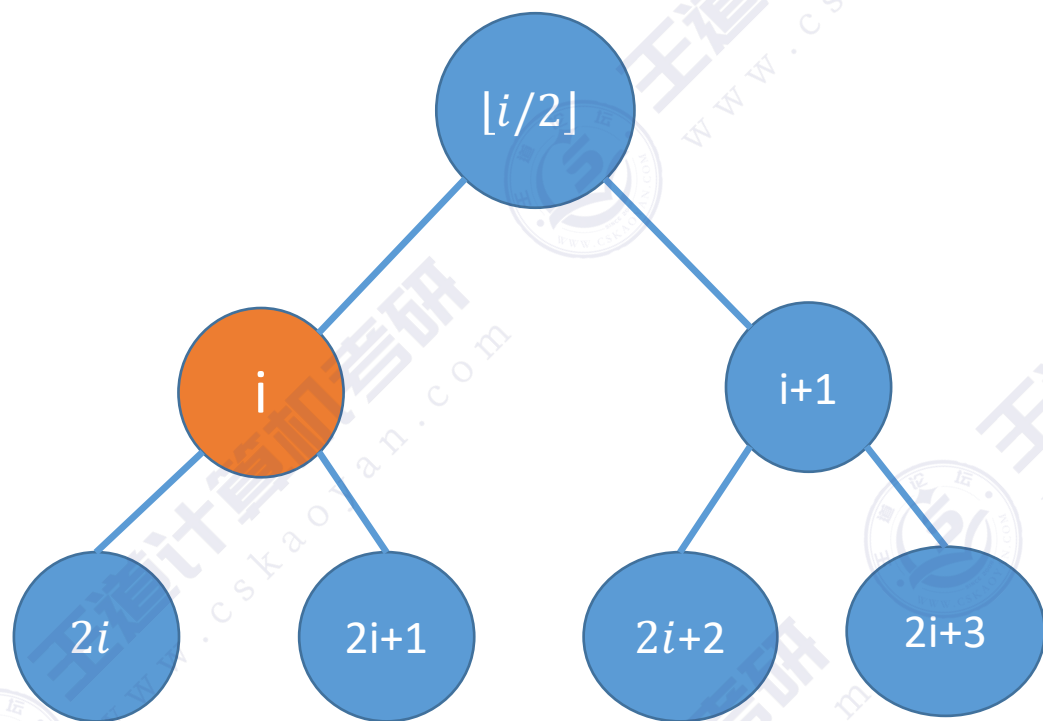
//二叉树的结点 (链式存储)

```
typedef struct BiTNode{
    ElemType data;
    struct BiTNode *lchild,*rchild;
    struct BiTNode *parent;
}BiTNode,*BiTree;
```

//数据域
//左、右孩子指针
//父节点指针

三叉链表——方便找父结点

知识回顾与重要考点



思考：如果结点从0开始编号，该怎么算



二叉树的顺序存储中，一定要把二叉树的结点编号与完全二叉树对应起来

- i 的左孩子 —— $2i$
- i 的右孩子 —— $2i+1$
- i 的父节点 —— $[i/2]$

最坏情况：高度为 h 且只有 h 个结点的单支树（所有结点只有右孩子），也至少需要 2^h-1 个存储单元

结论：二叉树的顺序存储结构，只适合存储完全二叉树

知识回顾与重要考点

//二叉树的结点 (链式存储)

```
typedef struct BiTNode{
```

```
    ElemType data;
```

```
    struct BiTNode *lchild,*rchild;
```

```
}BiTNode,*BiTree;
```

//数据域

//左、右孩子指针



n个结点的二叉链表共有 $n+1$ 个空链域

